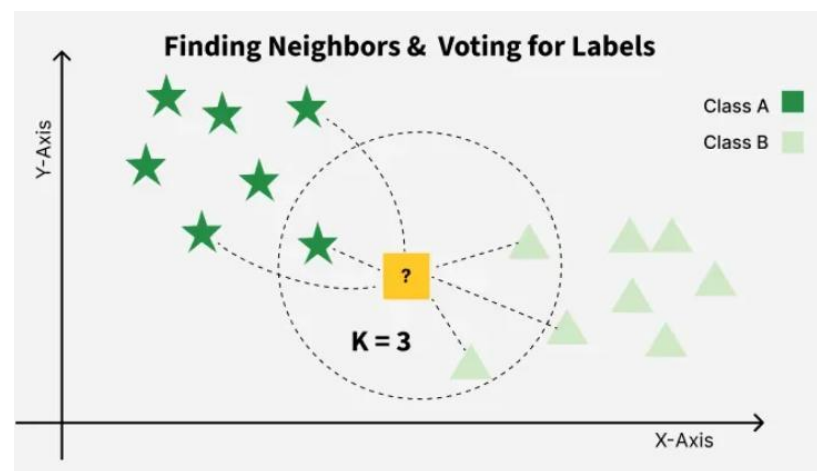
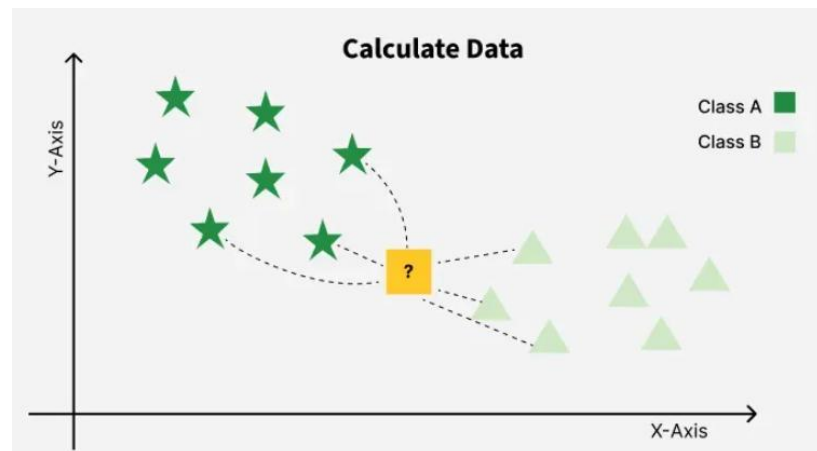


K-Nearest Neighbours(K-NN) algorithm



KNN Exercises

Exercise 01:

The training data in Figure 1 consists of four classes, namely $C = \{\text{red triangle, blue square, green star, black heart}\}$. No label is assigned to the data points A, B, C, D, E, and F, yet.

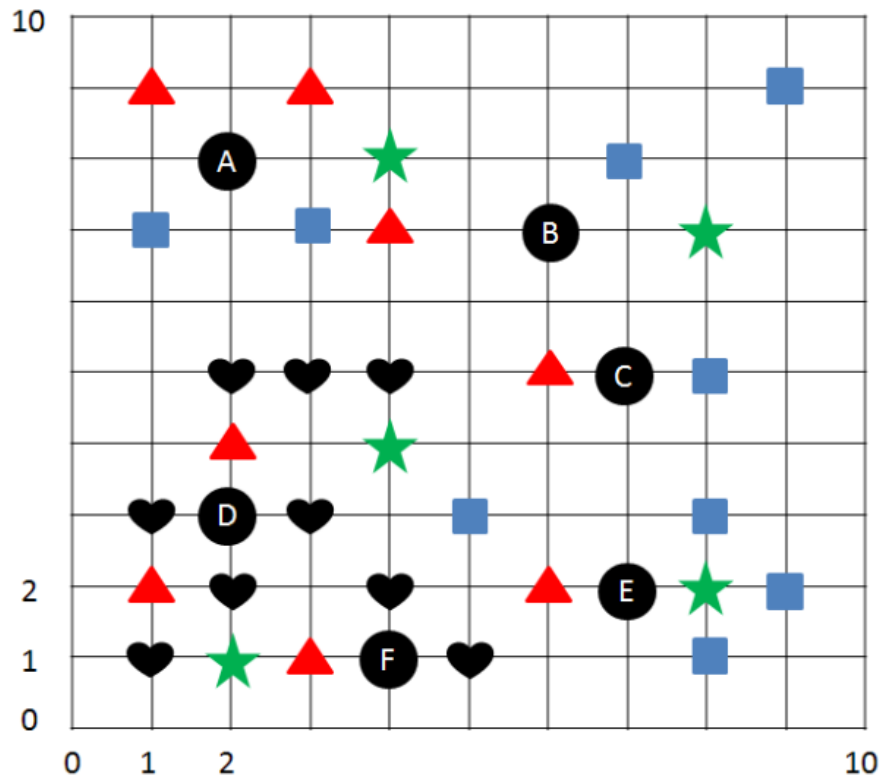


Figure 1: Data for k nearest neighbor classification

a. Construct a step-by-step distance calculation and classification table for each of the unlabeled points A, B, C, D, E, and F under the following six settings:

- Euclidean distance with $k = 4$
- Euclidean distance with $k = 5$
- Euclidean distance with $k = 6$
- Manhattan distance with $k = 4$
- Manhattan distance with $k = 5$
- Manhattan distance with $k = 6$

Note: Do not use the newly classified data points to classify any of the other data points. For instance, if you classify point B as a blue square, do not use this additional information for the classification of point C.

b. Provide a comprehensive comparative analysis of the classification results across all settings:

- Discuss the impact of varying the neighborhood size parameter k ($k=4, 5, 6$).
- Discuss the impact of varying the distance metric (Euclidean distance vs. Manhattan distance).

c. Write a Python program to implement the KNN algorithm from scratch based on the problem specification above:

- **Inputs:** Target point name ('A', 'B', 'C', 'D', 'E', or 'F'), value of k , and the distance metric ('euclidean' or 'manhattan').
- **Outputs:** Display detailed intermediate computation steps, including the sorted neighbor table, the selected k nearest neighbors, the vote distribution for each class, and the final predicted label.
- **Demonstration:** Execute and print the step-by-step console output for point E with $k = 5$ using both Euclidean and Manhattan distance metrics.

Note: Do not use built-in KNN implementations (e.g., from scikit-learn). You may only use standard numerical libraries (math, numpy, scipy.spatial.distance) and plotting libraries (matplotlib).